© 2021 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works. to appear in
IEEE 17th International Conference on Automation Science and Engineering (CASE), Lyon, France, 2021.

Performance Evaluation of Real-Time ROS2 Robotic Control in a Time-Synchronized Distributed Network

L. Puck[1] P. Keller[1] T. Schnell[1] C. Plasberg[1] A. Tanev[1] G. Heppner[1] A. Roennau[1] R. Dillmann[1]

***Abstract*— Modern robots are mainly controlled by monolithic black-box controllers provided by the individual manufacturers. In research institutions the first version of the Robot Operating System (ROS1) is widely used for different applications. However, ROS1 lacks real-time capable communication. The ongoing development of ROS2 promises to break this paradigm. By employing Data Distribution Service (DDS) as a middleware the modular architecture aims at providing real-time capabilities. This study assesses the current prospects and limitations of ROS2. It gains novel insights towards improved and, in particular, reliable results regarding latencies and jitter. To this end, the allocation and transmission of ROS2 messages is evaluated in an example application for robotic control. An oscilloscope is applied for external validation of the measurements in such a time-synchronized distributed network. The complete application is set up from non-real-time object detection towards real-time control via ROS2 and EtherCAT. An in-depth evaluation of the ROS2 communication stack on a single host and in distributed setups is included. With real-time safe memory allocation and highly privileged ROS2 processes real-time capabilities are ensured.**

I. INTRODUCTION

Robotic manufacturers commonly provide monolithic black-box controllers for their industrial robots. Internally they consist of a highly optimized system to ensure real-time functionality. Multi-component robots in industry and research, like walking robots or autonomous vehicles, would benefit from distributed controllers and additionally connected perceptual components. Here, real-time communication is the key to provide similar performance as one central system. Those new modular robotic controllers could potentially be based on the second version of the Robot Operating System (ROS2) [1]. To this end, an in depth evaluation of ROS2 publishers and subscribers on a time-synchronized system is conducted (see Figure 1). The results are assessed with an external measurement of response time, latencies and jitter. Therefore, this study guides through the steps taken to ensure undisturbed real-time capable ROS2 applications.

Due to employing DDS (Data Distribution Service) [2] as real-time capable middleware, ROS2 [1] arouses interest both in academia and commercial applications. The previous version ROS1 [3] relies on TCPROS and UDPROS based on the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP). In contrast to ROS1, ROS2 promises real-time capabilities through leveraging DDS. The development of ROS1 and ROS2 are based on its open-source community.

[1] Department of Interactive Diagnosis and Service Systems (IDS), FZI Research Center for Information Technology, Haid-und-Neu-Straße 10–14, 76131 Karlsruhe, Germany.

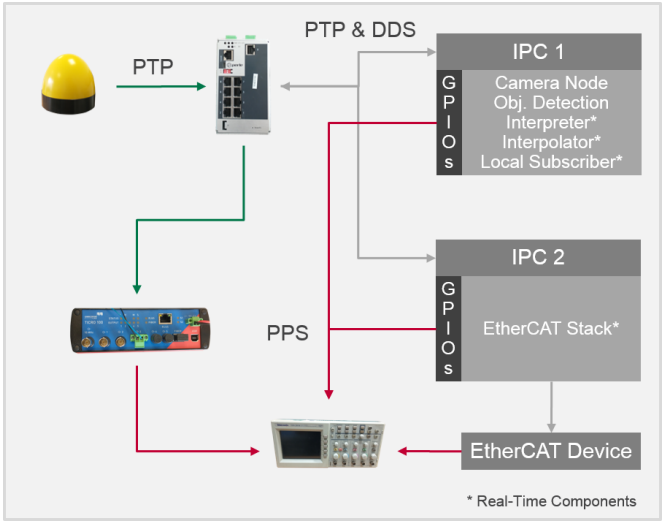


Fig. 1. The distributed robotic control system setup is divided onto two PCs (IPCs), both running ROS2 Eloquent Elusor, synchronized using the precision timing protocol (PTP). The PTP signal transmits the reference time for all network participants. On IPC1 run the non-real-time nodes of the camera node and for the object detection. Additionally, the three real-time components interpreter, interpolator and local subscriber, for evaluation purposes, are executed. On IPC2 the EtherCAT master is deployed, which serves as remote subscriber in the evaluation. All components trigger General Purpose Input/Output (GPIO) signals for latency measurements with the oscilloscope.

Regular releases integrate the newest features and improvements. Furthermore, different DDS implementations, both royalty free and commercial, can be used interchangeably with ROS2.

Guiterréz et al. [4] conducted a thorough analysis of ROS2 and different DDS vendors. The authors further assessed the influence of CPU load and network traffic. Moreover, they propose methods to achieve bounded latencies. Response time of processing chains has been evaluated by Casini et al. in [5]. The authors display a real-time scheduling model for ROS2 and further provide an analysis to measure the worst-case latencies of a robotic application.

Current research highlights the communication latencies of ROS2 and the influence of both CPU and network load. However, an extensive evaluation with regards to real-time capabilities is missing. Therefore, this study uses the precision timing protocol (PTP) to synchronize a previously established real-time capable system. Effects of different allocation strategies of ROS2 messages are assessed from a real-time perspective. Furthermore, the latencies and jitter across local and distributed ROS2-nodes are evaluated and

validated using external measurements with an oscilloscope.

This work contributes a thorough evaluation of ROS2 from a real-time perspective. A demonstrator is set up, which contains a ROS2 software stack, from object detection without real-time capabilities, across real-time ROS2 communication towards EtherCAT signals. The focus lies on the ROS2 real-time communication. Therefore, in-depth measurements of communication both across network and thread boundaries are undertaken. To this end, the triggered General Purpose Input/Output (GPIO) signals from the ROS2 applications allow for an external high-precision measurement of the latencies with respect to the PTP reference time with an oscilloscope.

This approach targets a ROS2 communication setup with a communication frequency of at least 1kHz. Furthermore, hard real-time-requirements should be fulfilled, however, no mathematical proof will be undertaken. Instead, long-term evaluations are executed, examining stability over a time frame of one week. By conducting these evaluations, modular components of real-time robotic controllers become feasible. Thereby allowing to reuse the modules for different robots.

The paper is structured as follows:

- In Section II a review of the current research will be presented.
- Section III introduces the hardware and software setup of the demonstrator.
- Section IV is divided into two parts. First, different allocation strategies of ROS2 messages are inspected. Afterwards, the complete communication setup is analysed with regards to real-time capabilities.
- In Section V the contributions are highlighted and discussed.
- Finally, in Section VI a conclusion and an outlook are given.

II. STATE OF THE ART

ROS1 [3], previously mainly used in research applications, is lacking one of the fundamental aspects to be considered for advanced industrial applications: Real-time capability. The successor ROS2 [1] has in its core design principles the integration of the Data Distribution Service (DDS) [2] as a real-time capable middleware [6]. This potential opens new research and application possibilities. Therefore, ROS2 has been studied intensively from the first releases.

One of the initial evaluations is conducted by Maruyama et al. in [7]. In their research the authors evaluate the real-time capabilities of ROS1 and ROS2 both in local and remote cases. Furthermore the authors employ three different DDS implementations (Connext [8], OpenSplice [9] and FastRTPS [10]). Their findings show that OpenSplice even outperformed Connext with enabled shared memory on a local system. The authors apply different quality of service profiles for each DDS. They further state that they can cannot find differences in latencies between ROS1 and ROS2 communication. However, ROS2 communication has an equal distribution of latency for multiple subscription nodes. The evaluation by Maruyama et al. [7] is based on one of the early alpha releases, since then both ROS2 and the DDS frameworks have undergone further improvements.

Gutiérrez et al. [11] first assess the network communications on Linux. They show that by employing the Preempt-RT patch the determinism of the communications is greatly improved. Further evaluation of the performance of ROS2 is continued by Guitérrez et al. in [4]. For their evaluation they took three different DDS implementations, varying CPU load and network traffic into account. In the evaluation the authors measure round-trip times between a standard PC and an embedded dual core device. For a system under load with 1Mbps concurrent network traffic they measure latencies of up to 2182µs. On a stressed real-time system with no network traffic, their best DDS setup reaches 1823µs round-trip communication.

However, their evaluation lacks a profound analysis of the real-time capabilities of the underlying Preempt-RT system. Next to Gutiérrez et al. [11] the Linux preempt system and network communication is further evaluated by Dantam et al. [12], Reghenzani et al. [13] and Puck et al. in [14].

Finding a trade-off between security and performance is a non-trivial task as Kim et al. [15] state. In their evaluation they show an in depth evaluation of different DDS security implementations and the respective throughput and latencies. During their study they found that ROS2 in their evaluated version does not adhere to the Object Management Groups (OMG) security specifications. However, since ROS2 is under heavy development this might change with future releases. For wired networks, with security disabled, their evaluations shows estimated latencies around $700 - 1000$µs for varying packet sizes.

Casini et al. [5] perform a response-time analysis for processing chains. Their work provides a first step to automated analysis of latency and temporal safety properties of their application. Furthermore, they inspect the scheduling policies of ROS from a real-time perspective. The authors provide tooling to reason about the worst-case latencies of different package designs without adapting code, thereby, aiding developers in creating time-safe communication stacks beginning at design time.

Staschulat et al. [16] propose an ROS2 executor which allows deterministic scheduling for ROS2 applications. The authors show a proof of concept for a real-time system on a 32-bit microcontroller. The executor supports the domain specific requirements. An extended evaluation of the Callback-group-level executor is conducted by Yang et al. in [17]. Their evaluation of the real-time capable executor using a ping pong test show that this executor can promote real-time capabilities for ROS2.

Finally, Park et al. [18] conduct an in depth comparison of latencies and jitter between ROS1 and ROS2. Their latency test concludes a round-trip latency test taking the software timestamps of the messages. For varying message sizes of up to 10kB their maximum round-trip time consist of 0.4 to 0.9 ms with 100Hz communication in 30 minute long trials. Moreover, applying the system on a multiagent robot system for evaluation and thereby showing that ROS2 outperforms

ROS1 with regards to real-time constraints.

The previous work shows insights into ROS2 and its behavior in real-time applications. However, long term tests and evaluation of the underlying operating system are missing. In this work we examine the behavior of ROS2 Eloquent Elusor on a previous established real-time Linux system with Preempt-RT Patch. An in-depth analysis of memory allocation strategies of ROS2 messages is conducted. Furthermore, different strategies to ensure undisturbed real-time applications across a distributed network are applied.

III. SYSTEM SETUP

For the demonstrator a setup of consumer-based hardware is used. Figure 1 displays the general setting of the demonstrator. The core components are the two identical industrial PCs (IPCs). The hardware of the IPCs was chosen due to the promising results of the OSADL [19] real-time benchmarks. The IPCs communicate with each other using ROS2 and the underlying DDS OpenSplice. Furthermore, both IPCs are time-synchronized using the precision timing protocol (PTP). The time of the PTP Grandmaster is applied as the reference time for all later measurements. Setting up the real-time Debian system with Preempt-RT Linux Kernel on the IPCs was established in previous work [14]. Details of the hardware specifications are stated in Table I.

TABLE I
IPC SYSTEM SETUP

Hardware	
Platform	Shuttle SH370 R8
Chipset	Intel H370
Processor	Intel Core i9-9900K (3600 MHz, 8 Cores)
RAM	G.Skill DIMM 16GB DDR4-3200
Setup	
OS	Debian 10
Kernel (Preempt Patch)	4.19.72(-rt25)
BIOS	2.20.1271 (American Megatrends)
PTP	
Grandmaster	OTMC 100i (Omicron)
PTP time converter	TICRO 100 - OCXO 25 (Omicron)
Switch	IDS 509 (Perle)
Software	
ROS2	Eloquent Elusor
DDS	Vortex OpenSplice Community Edition

The software deployed on both IPCs is ROS2 Eloquent Elusor with Vortex OpenSplice Community Edition as DDS middleware. Moreover, on both IPCs the serial interface pins of the motherboard are mapped into user-space and used as GPIOs. Thereby, the pins can be triggered by setting the according register directly from the ROS2 application, while introducing only minimal overhead. Thus, the oscilloscope can measure the rising flanks for each GPIO. By setting this into relation to the reference PTP signal, the latencies are externally measured.

For the demonstrator a software stack of multiple ROS2 nodes is set up. IPC1 functions as data processing unit, whereas IPC2 performs the communication with the EtherCAT devices. This leads to two non real-time nodes on IPC1, the camera node and the object detection node, which calculates a target position in world coordinates. This target position is further processed by computational real-time components, namely the interpreter and interpolator. The interpreter transforms the coordinates into robot coordinates, whereas the interpolator calculates the trajectory towards the goal. The results of the interpolator are then identically published on two topics with an offset of 150µs. The first topic is for the distributed setup. The second one, with an offset to avoid racing conditions, is utilized for local evaluation. For the latter, a local subscriber is started on ICP1. IPC2 only consists of the EtherCAT master, which sends the calculated target values to the EtherCAT devices. This ROS2 node is regarded as the remote subscriber for the evaluation.

Since this work focuses on the evaluation of the ROS2 real-time capabilities, the non-real-time nodes are disregarded for the further evaluation. Similarly, the EtherCAT communication from the EtherCAT master is not further evaluated. However, it is checked that the communication is indeed in real-time as EtherCAT is designed to be [20]. Thereby, the detailed evaluation is conducted between the interpolator, which publishes both for local and distributed communication. For each publisher, there is a corresponding subscriber. The publishers toggle their assigned GPIO pin directly before publishing and the subscriber as soon as the callback is executed. After a predefined active period the GPIOs are automatically toggled off.

IV. DISTRIBUTED REAL-TIME ROS2 COMMUNICATION

The following section first introduces the steps to ensure that the allocation of the message subscription is real-time capable. Afterwards the communication is further evaluated and improved in both networks, the local and the distributed one.

A. ROS2 Evaluation

The communication of ROS2 is designed as a publisher-subscriber system, whereas each node can function as both. Furthermore, each node can handle multiple publishers and subscriptions. A provided message can directly be published by the node, the subscriber has to check if a message arrives and then fill a datastructure with the contents. Once the node is notified about an incoming message the subscription can be divided into three parts. Figure 2 highlights the major parts of the reception while analyzing the function stack using *ros2_tracing* [21]. The Allocator assigns memory for the arriving message, RCL Take fills the allocated memory with the received values. Finally, the Handle Message executes the callback and further processing can be conducted. From the function stack it is visible, that the memory allocation requires most of the computation time.

Dynamic memory allocation is generally not real-time safe and upper bounds have to be evaluated for each task [22]. Therefore, the standard memory allocator employed by ROS2 might not act deterministic and thereby is not real-time safe. However, ROS2 allows to change the allocator. By choosing the Two-Level Segregated Fit allocator (TLSF)

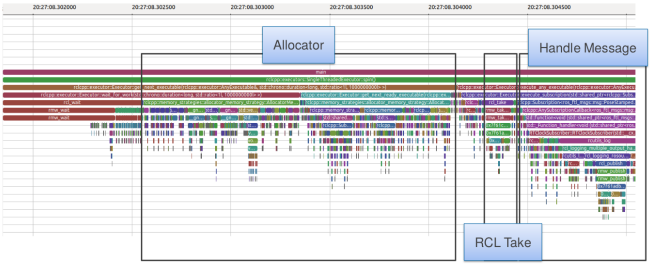


Fig. 2. Analysis of the function trace call using *ros2_tracing* [21]. The function call can roughly be divided in three major parts. The Allocator is responsible for allocating the memory for the message. RCL Take fills the provided memory with the actual contents of the message. Finally, the callback will be executed in the Handle Message. When conducting the analysis multiple times, the Allocator generally needs the largest time slice. However, each cycle takes a varying amount of time, thereby acting non-deterministic in the default setup.

[23], a real-time safe implementation is deployed. Table II displays the differences of memory allocation between the default and the TLSF allocator. The in section IV-B described real-time safe setup for local communication is used to evaluated the different allocation strategies. The default allocator requires in the worst-case more than 1ms. Thereby, being slower than the expected cycle time, whereas, the TLSF allocator only takes 144.4µs. If additional pooling of pre-allocated messages is added, the worst-case drops slightly further to 136.6µs. Furthermore, the mean time of the allocation process is decreased to 78.5µs.

TABLE II
ALLOCATOR STRATEGIES

	Default Allocator	TLSF Allocator	TLSF + Pooling
Min	68.7 µs	68.8 µs	71.0 µs
Mean	88.4 µs	86.5 µs	78.5 µs
Max	>1.0 ms	144.4 µs	136.6 µs

The current implementation used in ROS2 has some drawbacks however. It does not allow for the usage of unbound strings in the message itself, thereby making all stamped ROS2 messages obsolete in real-time ROS2 applications. This is due to fact that the default message header for stamped messages integrates strings. This could be solved by making strings by default bounded The same issue arises for arrays and other dynamic datastructures inside the messages.

ROS2 further allows the usage of custom executors. Thus, the default executor was stripped to only handle the message subscription and ignore all other tasks. This however only led to a marginal improvement in the application and was therefore not further pursued.

B. Communication Evaluation

The evaluation of the ROS2 communication consists of the transmission between the publisher and subscriber. It is measured by taking the rising flanks of the GPIOs using the oscilloscope. The evaluation is conducted both for local transmission using the loopback device and for remote communication via network communication. Intraprocess communication is not further evaluated since, for our purpose, the different components should be independent and the communication in-between them real-time capable. In the commercial DDS implementations there are settings to use shared memories. However, the royalty free version of Vortex OpenSplice Community Edition, which is used for this evaluation, does not provide this option.

The communication timings are visualized in Figure 3. Each cycle continues for 1ms, thereby running in a desired frequency of 1kHz. The remote publisher is scheduled to wake up immediately at the beginning of the cycle, the local one with 150µs offset. For the rising flanks of the publishers the response latency and jitter are measured. In Figure 3 the shaded area around the flanks symbolizes the jitter in the latencies. For each signal t_0 describes the minimum and t_1 the maximum measured latency. For the communication the difference between the rising flanks of publisher to subscriber is measured.

In Figures 4-7 the experimental results are highlighted. Since the measurements are recoreded with an oscilloscope it might happen that a flank is set to high at the beginning of the loop. This can be due to the timings being off, or messages arriving too late. Both resulting in a bad communication and potentially lost messages. Furthermore, the results are shown as direct results from the oscilloscope. The sampling rate of the KeySight MSOS604A Oscilloscope for frequencies of 1kHZ is used without any adjustments. Due to the sampling rate the counter of measurements appears as a factor of the actual send messages. However, since there have been no modifications and adjustments to the sampling rate the results are applicable.

To establish and minimize the latencies of communication different strategies are applied to the ROS2 stack. First, a baseline measurement on a stressed[1] environment is established. Afterwards, the different strategies will be presented step by step with the according evaluation alongside them.

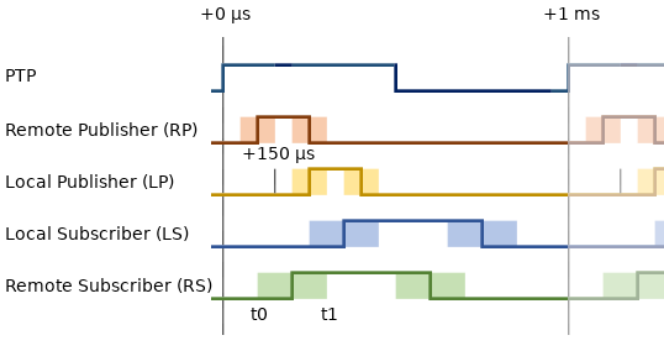


Fig. 3. The timing chart of the evaluation setup. The PTP cycle time is 1ms. At the beginning of each cycle the remote publisher is scheduled to wake up. The local publisher is scheduled with an offset of 150µs For both the publishers and the subscribers the rising flank is measured. In lighter coloring the jitter of each flank is visualized. The variable t_0 describes the minimum and t_1 the maximum latency. In Figures 4-7 these describe the difference between the earliest rising flank and the latest.

[1]Command used: *stress -c 8 -i 8 -m 8 -d 8*

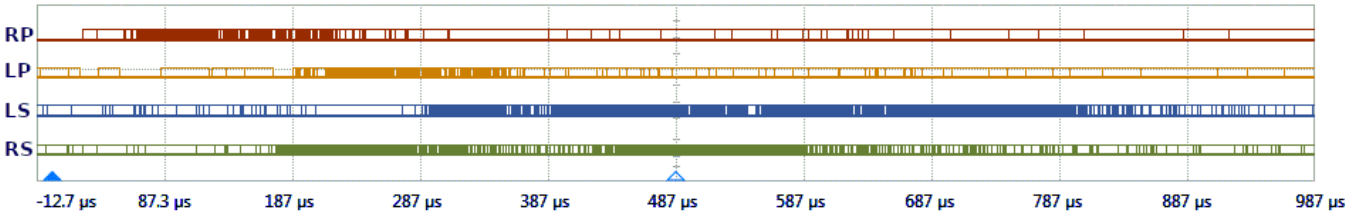


Fig. 4. The baseline evaluation run conducted over 10^6 measurements under a stressed system. Clearly, the response time of the publishers is off, thereby the whole system fails any real-time requirements. Ideally the red and yellow curves would resemble the corresponding publishing timings of Figure 3. (Key: (R)emote/(L)ocal and (P)ublisher/(S)ubscriber)

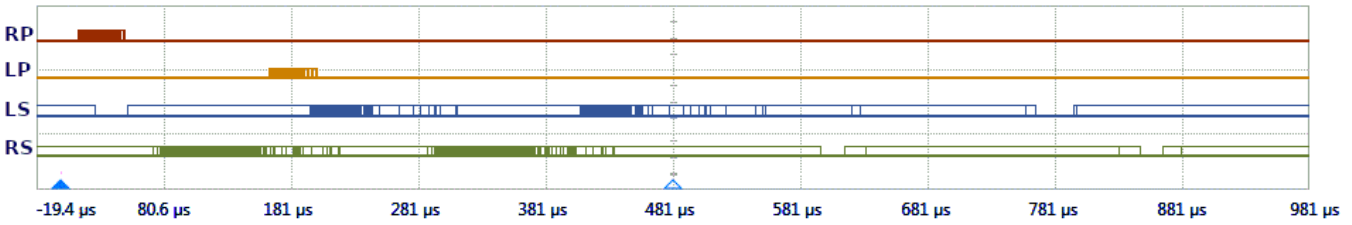


Fig. 5. With real-time priority of the ROS2 nodes and memory locking the publishing is performed in bounded time. However, this run of over 10^6 measurements was not performed in a stressed environment. The subscriptions (blue and green) show delays and lost measurements, therefore this part does not fulfill real-time requirements. In contrast to the baseline, the publishing is in bounded time. (Key: (R)emote/(L)ocal and (P)ublisher/(S)ubscriber)

Furthermore, the quality of service (QoS) profile for ROS2 is set to the sensor QoS profile. This is due to the profile being non-blocking since the best effort policy is applied. However, the queue size is reduced to 1, to only allow for the latest message to be stored to not add additional delays and rather discard old messages.

The effects of applying stress to both IPCs is displayed in Figure 4. The results for the subscription can be neglected since the publishing is already not executed with the desired frequencies. Thereby, it is clearly stated that the ROS2 nodes do not respond in the desired time-frame. Therefore, the first step is to ensure, that the node itself has the correct real-time priority. This is done by setting the priority directly from the code, however this only sets the priority for the started node and does not include the corresponding DDS threads in the background. Furthermore, to avoid dynamic memory allocation, memory is pre-allocated and mapped. However, pre-allocating memory has the risk of allocating too much memory, possibly leading to a huge overhead of unused memory. It is best used in applications with a reasonable small known memory footprint. The results are displayed in Figure 5 in an unstressed environment. Publishing the messages is now conducted in a bounded time-frame each cycle. The maximum response time is 31.0µs. However, the subscription still does not react accordingly and some messages do not arrive during the cycle time and are therefore lost.

The delayed subscription are examined through an analysis of the involved processes. This clarifies that next to the ROS2-Node, the *xmit*-thread of the DDS middleware OpenSplice, which is responsible for the transmission, has frequent computation time. By just setting the node to real-time priority the OpenSplice threads do not inherit the same priority. Thereby, all the corresponding threads of a node have to manually be configured to use real-time priority[2]. This leads to the results in Figure 6. The test is again conducted in a stressed environment. The response time of the publishers now is appropriate. The subscriptions, especially for the local subscription are bounded with the maximum time of 173.3µs. For the remote subscription the latencies are higher with 466.9µs. This is assumed to mainly caused by the network communication across the Linux network stack.

TABLE III
TRANSMISSION LATENCIES IN µs

		Idle	Stress	Isolation	Shielding
LOCAL	**Min.**	71.0	68.6	186.8	67.0
	Mean	78.6	85.9	198.5	79.9
	Max.	136.6	173.3	280.8	192.4
REMOTE	**Min.**	81.6	82.4	–	81.0
	Mean	105.2	113.7	–	108.7
	Max.	240.9	466.9	–	421.2

Further decrease of jitter and latencies is achieved by shielding half of the cores. Four cores are then solely used for the ROS2 and OpenSplice communication. Thereby, the stress on the system does not influence the nodes and transmission on application level. In contrast to isolation of CPUs, shielding keeps essential core threads are running on the CPUs and moreover the load balancing is still working [14]. The final evaluation was conducted over one week with over 12×10^6 measurements of the oscilloscope. Despite this long and continuous run, no messages were lost or arrived outside the cycle time of 1ms. Thereby, the setup appears to be robust and real-time capable over a long period of time. The maximum latency in local transmission is 192.4µs, remote transmission takes longer with a maximum of 421.2µs.

[2]Command used: *sudo chrt -a -f -p 95 <PID>*

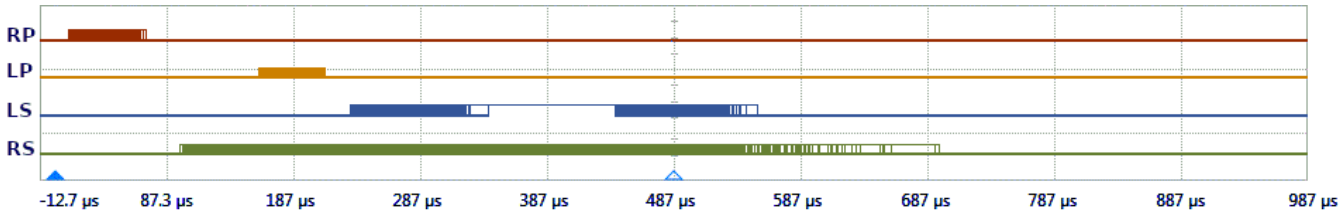


Fig. 6. Correctly set real-time priorities for all involved threads. Especially the setting of the ROS2 and the *xmit*-OpenSplice threads, allow for both publishing and subscribing in sufficient time. The test of over 10^6 measurements is conducted in a stressed environment. All four curves resemble the ideal curves presented in Figure 3. (Key: (R)emote/(L)ocal and (P)ublisher/(S)ubscriber)

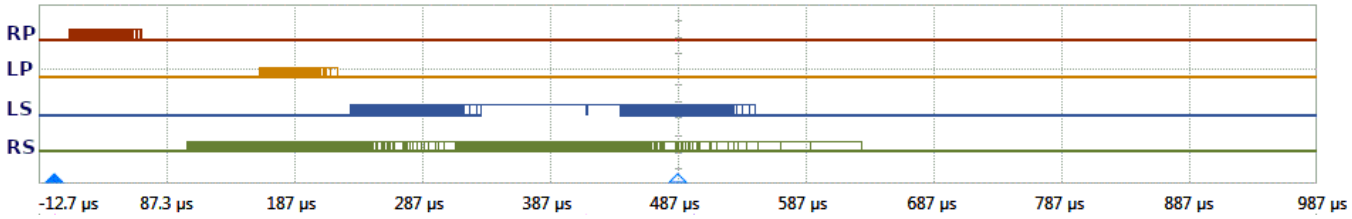


Fig. 7. Employing all previous introduced measurements allow for a real-time ROS2 application. By shielding the real-time critical components further decreases in the latencies are achieved. The final run was executed over 12×10^6 measurements for one week. Thereby showing that the results hold true over a longer time frame. (Key: (R)emote/(L)ocal and (P)ublisher/(S)ubscriber)

Further results are shown in Table III. Due to isolation leading to unexpected desynchronized behavior, there are no measurements for the remote case. The Idle, Stressed and Isolation case consist of over 10^6 measurements, whereas the Shielding case contains over 12×10^6 measurements.

Depending on the steps of ROS2 communication different cycle frequencies can be achieved. In the presented demonstrator's stack, the real-time communication is between the Interpreter/Interpolator and the Interpolator/EtherCAT master. Thereby, a two-step communication setup is able to run at a frequency of up to 1kHz.

V. RESULTS

In this work the real-time capabilities of ROS2 are evaluated on off-the-shelf consumer hardware with an optimized configuration on the software side. The Linux systems use the Preempt-RT patch to achieve real-time behavior comparable to leading systems in the OSADL benchmarks. Presumably exceeding the systems of Gutiérrez et al. [4] and Casini et al. [5] in terms of real-time properties. Both apply the Linux Preempt-RT patch as well, however it is not further evaluated if any idle states are entered or low-level power-saving features interfere with the system. Previous work [14] eliminates these idle states, configures a reliable Linux system. This shows a significant performance boost without any delayed system responses, especially since these delayed responses only occur rarely on an otherwise idle system. Thereby, the magnitude of latencies measured in this research are solely due to the ROS2 and DDS communication via the Linux network stack.

Park et al. [18] prove that 0.31 to 0.63ms latencies can be reached with round-trip measurements on idle systems with no network traffic. Adding 30Mbps slows down the communication to 0.58 to 0.83ms for various package sizes. Comparing their work to ours, their system reaches similar latencies for an idle system. However, Park et al. only record measurements of 30 minutes on an unstressed system. Thereby, it is to be expected, that their results would increase when employing stress. Furthermore, these short tests potentially miss the rarely occurring delays, which where experienced during this study. On the other hand, they have integrated network traffic into their evaluation. This is disregarded in this work due to the desynchronization of the PTP synchronization, which leads to non-comparable results. To further evaluate this, a separate and dedicated network for the PTP communication should be set up. Through a separated network the precise synchronization could be ensured through stable round-trip times, while simultaneously allowing higher traffic on the network dedicated for communication.

Key findings of this study are:

- **Employing real-time safe dynamic memory allocation.** The standard allocator used in ROS2 is not real-time ready. However, using the TLSF allocator [23] allows the subscription to be in bounded time. Pre-allocating messages further decreases the latencies of the subscription. Additionally, if the memory footprint is known, memory can be pre-faulted and locked for the ROS2 application. However, this is only feasible for nodes with a small memory footprint.
- **Shielding real-time prioritized processes.** All processes have to run with real-time priority, otherwise there might be surprises with delayed messages. Especially, since the DDS middleware spawns additional processes which have to be taken into account. Additionally, shielding the processes decreases disturbances and allows for a more robust real-time communication. Thereby, even on an otherwise stressed system, the ROS2 and DDS processes can be real-time safe.

- **Deploying real-time ROS2 applications.** Communication across network boundaries is achieved with a maximum latency of 421µs. Thereby, staying under the desired control frequency of 1kHz. Local ROS2 networks using the host's loopback device reduce the latencies by a factor of two, potentially doubling the control frequency.

VI. CONCLUSIONS AND FUTURE WORK

This study investigates the real-time capabilities of ROS2. Long-term experiments suggest that ROS2 indeed meets hard real-time requirements with the correct system configuration. Sufficient steps to run real-time ROS2 applications on consumer-based based hardware are given. The communication latencies were measured using an oscilloscope, for both local and distributed communication. The latencies are thereby validated externally and the results hold true over a longer time frame.

A ROS2 communication stack was set up from non-real-time camera input, across real-time ROS2 communication towards EtherCAT through a distributed real-time system. An in-depth evaluation of the ROS2 communication across a distributed system was conducted. The latencies between the shielded ROS2 nodes are under 422µs across the network boundary and below 193µs across thread boundaries. Holding true during a trial of 7 days with over 12×10^6 external measurements. Thereby, allowing the whole setup with two communication steps being able to run at 1kHz. Furthermore, shielding was employed to further mitigate the effect of other perturbations.

The results showing the real-time potential of ROS2, while simultaneously revealing weaknesses that are still present. The results allow for the development of an actual modular robotic controller, thereby breaching the monolithic design principle. The extensive measurements show the feasibility of a real-time ROS2 system, even through long time evaluations with external measurements reliable results could be obtained.

In future work, different DDS Versions (e.g. Eclipse Cyclone DDS [24]) and the corresponding settings for real-time should be evaluated. Furthermore, the main limitation is the Linux network stack in remote communication. Therefore, alternatives such as XDP [25] could improve the real-time behavior. Additionally, the default executor could be exchanged to the proposed rclc executor proposed by [16]. This could further benefit the real-time behavior of a complete ROS2 stack.

REFERENCES

[1] "ROS2," visited on 2020-04-02. [Online]. Available: https://index.ros.org/doc/ros2/

[2] "DDS Interoperability Wire Protocol," visited on 2020-04-02. [Online]. Available: https://www.omg.org/spec/DDSI-RTPS/

[3] M. Quigley, B. Gerkey, K. Conley, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, and A. Ng, ""ROS: an open-source Robot Operating System"," in *Proc. of the IEEE Intl. Conf. on Robotics and Automation (ICRA) Workshop on Open Source Robotics*, Kobe, Japan, May 2009.

[4] C. S. V. Gutiérrez, L. U. S. Juan, I. Z. Ugarte, and V. M. Vilches, "Towards a distributed and real-time framework for robots: Evaluation of ROS 2.0 communications for real-time robotic applications," *arXiv preprint arXiv:1809.02595*, 2018.

[5] D. Casini, T. Blaß, I. Lütkebohle, and B. B. Brandenburg, "Response-Time Analysis of ROS 2 Processing Chains Under Reservation-Based Scheduling," in *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2019.

[6] J. Kay and A. R. Tsouroukdissian, "Real-time control in ROS and ROS 2.0," *Open Source Robotics Foundation*, 2015.

[7] Y. Maruyama, S. Kato, and T. Azumi, "Exploring the performance of ROS2," in *Proceedings of the 13th International Conference on Embedded Software - EMSOFT '16*. New York, New York, USA: ACM Press, 2016, pp. 1–10. [Online]. Available: http://dl.acm.org/citation.cfm?doid=2968478.2968502

[8] "Connext DDS Professional by RTI," visited on 2020-04-02. [Online]. Available: https://www.rti.com/products/connext-dds-professional

[9] "Adlink Vortex OpenSplice," visited on 2020-04-02. [Online]. Available: https://www.adlinktech.com/en/vortex-opensplice-data-distribution-service.aspx

[10] "eProsima FastRTPS," visited on 2020-04-02. [Online]. Available: https://www.eprosima.com/index.php/products-all/eprosima-fast-rtps

[11] C. S. V. Gutiérrez, L. U. S. Juan, I. Z. Ugarte, and V. M. Vilches, "Real-time Linux communications: an evaluation of the Linux communication stack for real-time robotic applications," *arXiv preprint arXiv:1808.10821*, 2018.

[12] N. T. Dantam, D. M. Lofaro, A. Hereid, P. Y. Oh, A. D. Ames, and M. Stilman, "The Ach Library: A New Framework for Real-Time Communication," *IEEE Robotics Automation Magazine*, vol. 22, no. 1, pp. 76–85, 2015.

[13] F. Reghenzani, G. Massari, and W. Fornaciari, "The real-time linux kernel: A survey on preempt_rt," *ACM Computing Surveys (CSUR)*, vol. 52, no. 1, pp. 1–36, 2019.

[14] L. Puck, P. Keller, T. Schnell, C. Plasberg, A. Tanev, G. Hepper, A. Roennau, and R. Dillmann, ""Distributed and Synchronized Setup towards Real-Time Robotic Control using ROS2 on Linux"," in *IEEE Intl. Conf. on Automation Science and Engineering (CASE)*, 2020.

[15] J. Kim, J. M. Smereka, C. Cheung, S. Nepal, and M. Grobler, "Security and performance considerations in ROS 2: A balancing act," *arXiv preprint arXiv:1809.09566*, 2018.

[16] J. Staschulat, I. Lütkebohle, and R. Lange, "The rclc executor: Domain-specific deterministic scheduling mechanisms for ros applications on microcontrollers: work-in-progress," in *2020 International Conference on Embedded Software (EMSOFT)*. IEEE, 2020, pp. 18–19.

[17] Y. Yang and T. Azumi, "Exploring real-time executor on ros 2," in *2020 IEEE International Conference on Embedded Software and Systems (ICESS)*. IEEE, 2020, pp. 1–8.

[18] J. Park, R. Delgado, and B. W. Choi, "Real-Time Characteristics of ROS 2.0 in Multiagent Robot Systems: An Empirical Study," *IEEE Access*, vol. 8, pp. 154 637–154 651, 2020.

[19] "Open Source Automation Development Lab," visited on 2020-04-02. [Online]. Available: https://www.osadl.org

[20] D. Jansen and H. Buttner, "Real-time Ethernet: the EtherCAT solution," *Computing and Control Engineering*, vol. 15, no. 1, pp. 16–21, 2004.

[21] "ROS2 Tracing," visited on 2020-10-19. [Online]. Available: https://gitlab.com/ros-tracing/ros2_tracing

[22] I. Puaut, "Real-time performance of dynamic memory allocation algorithms," in *Proceedings 14th Euromicro Conference on Real-Time Systems. Euromicro RTS 2002*. IEEE, 2002, pp. 41–49.

[23] M. Masmano, I. Ripoll, A. Crespo, and J. Real, "TLSF: A new dynamic memory allocator for real-time systems," in *Proceedings. 16th Euromicro Conference on Real-Time Systems, 2004. ECRTS 2004.* IEEE, 2004, pp. 79–88.

[24] "Eclipse Cyclone DDS," visited on 2020-10-29. [Online]. Available: https://projects.eclipse.org/projects/iot.cyclonedds

[25] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The express data path: Fast programmable packet processing in the operating system kernel," in *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies*, 2018, pp. 54–66.